The Dictionary ADT

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Homework 2 Recap

Mutation testing

- About evaluating your *tests*, not your *implementation*
- Surviving mutant = missing tests
  - Does not mean your implementation has that bug!

- Common testing gap: testing for error cases
  - Need to make sure they don't accidentally produce results!

- Homework 2 had a small testing surface
  - Homework 3 (out now!) has a much larger one!
  - I.e., needs more tests!

Coming Soon: First Midterm

First Midterm: Thursday May 1st (9 days)

- **Why exams?**
  - Assignments assess the *applied* portion of the class
  - Exams assess the *conceptual* portion of the class
  - Both are important $\rightarrow$ we need both assessments
- Some of the work engineers do resembles exams!
  - Software engineering is *not* just writing programs!
  - Exploring designs, evaluating alternatives, etc.
    - Without spending time to fully work *all* of them out!
  - Evaluating pros and cons of different approaches
    - Brainstorming with colleagues
  - Requires thinking on your feet, juggling concepts, thinking about details and exceptions, etc.
    - **Just like exams!**

Coming Soon: First Midterm

First Midterm: Thursday May 1st (9 days)

- **Material**: Up to incl. graph basics (Chapter 10, worksheet), **but not** graph ADTs and data structures (Tue next week)
- **Format**
  - In person, on paper
  - In this room, at this time (except ANU students)
  - No notes, no electronics, no nothing
  - Will release a practice exam (but don't overfit to it!)

What is a dictionary?

Dictionaries map *keys* to *values*.

- If you have a key, you can look up the associated value.
- Can add and (usually) remove key-value pairs.
- Both keys and values can be of any type.

What is a dictionary?

Dictionaries map *keys* to *values*.

- If you have a key, you can look up the associated value.
- Can add and (usually) remove key-value pairs.
- Both keys and values can be of any type.

For example:

- The titular example: (language) dictionaries.
- If I have a copy of the Oxford English Dictionary, I can look up a word (key)...
- ...and find its definition (value).

What is a dictionary?

Dictionaries map *keys* to *values*.

- If you have a key, you can look up the associated value.
- Can add and (usually) remove key-value pairs.
- Both keys and values can be of any type.

For example:

- Say I want to call my friend Willie.
- If I have a phonebook (dictionary)...
- ...I can look up his name (key)...
- ...and get find his phone number (value).

What is a dictionary?

Dictionaries map *keys* to *values*.

- If you have a key, you can look up the associated value.
- Can add and (usually) remove key-value pairs.
- Both keys and values can be of any type.

For example:

- I want to represent the cities in each US state
  $\rightarrow$ dict with states as keys and **sets** of cities as values.
  - For example: Illinois $\rightarrow$ { Chicago, Evanston, Joliet, ... }
- Can combine ADTs! For non-toy problems, you usually need to. (We'll see more throughout the quarter.)
- Lookup is *one-way*: can ask which cities are in Illinois, but can't ask which state Chicago is in! (stay tuned)

What is a dictionary?

Dictionaries map *keys* to *values*.

- If you have a key, you can look up the associated value.
- Can add and (usually) remove key-value pairs.
- Both keys and values can be of any type.

- Dictionary is an ADT; many possible implementations!
  - And tons of different ones exist, for different use cases
  - One of the most widespread and useful ADTs
- We'll see **one** way to define the dictionary ADT
  - Lots of variants, with different operations, laws, ...
  - All with pros and cons
  - All share the essence of mapping keys to values

The Dictionary ADT: values and operations

Looks like: $\boxed{\{a{:}6, b{:}7, c{:}8\}}$

Signature:

```
interface DICT[K, V]: # key and value types up to client
    def mem?(self, key: K) -> bool?
    def get(self, key: K) -> V
    def put(self, key: K, value: V) -> NoneC
    def del(self, key: K) -> NoneC
```

- Keys must all be distinct
- Order of keys is not significant

The Dictionary ADT: examples

- I have an English dictionary and want to look up the definition of the word “aberrant”:

  $\{\ldots, \text{"}\textit{aberrant}\text{"}:\text{”Deviating from the usual.”}, \text{"}\textit{aberuncator}\text{"}:\ldots\}$

  `d.get("aberrant")` $\rightarrow$ “Deviating from the usual.”

The Dictionary ADT: examples

- I have an English dictionary and want to look up the definition of the word “aberrant”:

  $\{\ldots, \text{"}\textit{aberrant}\text{"}:\text{”Deviating from the usual.”}, \text{"}\textit{aberuncator}\text{"}:\ldots\}$

  `d.get("aberrant")` $\rightarrow$ “Deviating from the usual.”

- I make a new friend, Spencer, and add his number to my phone book:

  $\{\ldots, \text{"}\textit{Willie}\text{"}:\text{”***-***-***7”}, \ldots\}$

  `d.put("Spencer", "***-***-***4")`

  $\rightarrow$ $\{\ldots, \text{"}\textit{Spencer}\text{"}:\text{”***-***-***4”}, \text{"}\textit{Willie}\text{"}:\text{”***-***-***7”}, \ldots\}$

The Dictionary ADT: laws

$$\{\}\ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.\mathit{mem?}(k_i) \Rightarrow \texttt{True}\ \{\}$$

$$\{\forall i, k_i \neq k\}\ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.\mathit{mem?}(k) \Rightarrow \texttt{False}\ \{\}$$

$$\{\}\ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.\mathit{get}(k_i) \Rightarrow v_i\ \{\}$$

$$\left\{d = \boxed{\{\ldots, k_i{:}v_i, \ldots\}} \wedge \forall i, k_i \neq k\right\}$$
$$d.\mathit{put}(k, v) \Rightarrow \texttt{None}$$
$$\left\{d = \boxed{\{\ldots, k_i{:}v_i, \ldots, k{:}v\}}\right\}$$

$$\left\{d = \boxed{\{\ldots, k_{i-1}{:}v_{i-1},\quad k_i{:}v_i,\quad k_{i+1}{:}v_{i+1}, \ldots\}}\right\}$$
$$d.\mathit{del}(k_i) \Rightarrow \texttt{None}$$
$$\left\{d = \boxed{\{\ldots, k_{i-1}{:}v_{i-1},\quad k_{i+1}{:}v_{i+1}, \ldots\}}\right\}$$

- That's a lot of stuff. We'll do one operation at a time.

Law breakdown: *mem?*

If the key we are looking for is present, we get True:

$$\{\}\ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.\mathit{mem?}(k_i) \Rightarrow \texttt{True}\ \{\}$$

If the key we are looking for is not equal to any of the keys in the dictionary, we get False:

$$\{\forall i, k_i \neq k\}\ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.\mathit{mem?}(k) \Rightarrow \texttt{False}\ \{\}$$

Neither of these laws care about the order of the keys!

Law breakdown: *get*

If we try to look up a key present in the dictionary, we get its associated value:

$$\{\} \ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.get(k_i) \Rightarrow v_i \ \{\}$$

Law breakdown: *get*

If we try to look up a key present in the dictionary, we get its associated value:

$$\{\} \ \boxed{\{\ldots, k_i{:}v_i, \ldots\}}.get(k_i) \Rightarrow v_i \ \{\}$$

If we try to look up a key that is not in the dictionary

- I.e., there is no k in the dictionary equal to k_i
- The laws don't specify behavior in that case
- Different languages/libraries do different things (more later)

Law breakdown: *put*

If we insert a key that's not in the dictionary, the new key and value association is added:

$$\left\{ d = \boxed{\{\ldots, k_i{:}v_i, \ldots\}} \wedge \forall i, k_i \neq k \right\}$$

$$d.put(k, v) \Rightarrow \text{None}$$

$$\left\{ d = \boxed{\{\ldots, k_i{:}v_i, \ldots, k{:}v\}} \right\}$$

If we insert a key that's already present, all bets are off!

Law breakdown: *del*

If we delete a key that's present, it gets removed:

$$\left\{ d = \boxed{\{\ldots, k_{i-1}{:}v_{i-1},\quad k_i{:}v_i,\quad k_{i+1}{:}v_{i+1}, \ldots\}} \right\}$$

$$d.del(k_i) \Rightarrow \text{None}$$

$$\left\{ d = \boxed{\{\ldots, k_{i-1}{:}v_{i-1},\quad k_{i+1}{:}v_{i+1}, \ldots\}} \right\}$$

If we delete a key that's not in the dictionary, all bets are off!

The Dictionary ADT: Variants

Diff languages/libraries can provide diff (or more!) operations!

- All with the same basic idea: key-value associations

Here are some common variations:

`get` of an absent key: $\boxed{\{Guava : Fruit\}}.get(\text{Cherry})$

- Raise an error.
  - Requires calling `mem?` before `get` (asking permission)
  - Or exception handling (asking forgiveness)
- Return a failure result (e.g., None)
  - How to distinguish from legitimate value of None?
  - Known as the *semipredicate problem*
- Call a client-provided failure callback function
  - Client can choose between raising an error, using some default value, etc.
  - Friendliest in my opinion

The Dictionary ADT: Variants

Diff languages/libraries can provide diff (or more!) operations!

- All with the same basic idea: key-value associations

Here are some common variations:

`put` of an existing key: $\boxed{\{Tomato : Veg\}}.put(\text{Tomato, Fruit})$

- Could update value
  - Convenient, `put` does double duty
  - Could be error prone: accidental key reuse does not error
- Or could throw an error
  - Catches key reuse mistakes
  - Needs a dedicated `update` operation

The Dictionary ADT: Variants

Diff languages/libraries can provide diff (or more!) operations!

- All with the same basic idea: key-value associations

Here are some common variations:

`del` of absent keys: $\boxed{\{Guava : Fruit\}}.del(\text{Cherry})$

- Can ignore and do nothing
- Can throw an error
- Not all dictionaries even **have** a `del`!

Stop! Question time!

Before we move on to a common dictionary usage pattern...

Anything unclear so far?

Anything I missed?

Bidirectional mappings

Very common in programming. Dictionaries can help!

Want to look things up in two directions:

- Letters A–Z to numbers 1–26
  - Convert back and forth between letters and numbers
- Runners to their position in the race
  - May want to know which runner is in second place, **and** know which position runner Isabel is at
- US states to cities in them (revisited)
  - Now we can know which cities are in Illinois
  - And **also** know which state Chicago is in

Bidirectional mappings

Very common in programming. Dictionaries can help!

Want to look things up in two directions:

- Letters A–Z to numbers 1–26
  - Convert back and forth between letters and numbers
- Runners to their position in the race
  - May want to know which runner is in second place, **and** know which position runner Isabel is at
- US states to cities in them (revisited)
  - Now we can know which cities are in Illinois
  - And **also** know which state Chicago is in

Represent using a pair of dictionaries! One for each direction.

$\{A{:}1, B{:}2, C{:}3, \ldots\}$ and $\{1{:}A, 2{:}B, 3{:}C, \ldots\}$

Want to go from letters to numbers? Use the first dictionary!

Want to go from numbers to letters? Use the second dictionary!

The quest for a dictionary data structure

ADT vs Data Structure

- Dictionary is an *ADT*
  - Like stack, queue, sequence, set
  - Unlike linked list, ring buffer, dynamic array, etc.
    - Those are *data structures*

ADT vs Data Structure

- Dictionary is an *ADT*
  - Like stack, queue, sequence, set
  - Unlike linked list, ring buffer, dynamic array, etc.
    - Those are *data structures*

- We know how dictionaries should *behave*
  - I.e., their *specification*

- We still don't know we we could *build* one, though
  - I.e., their *implementation*: a concrete data structure

- Let's hear your ideas

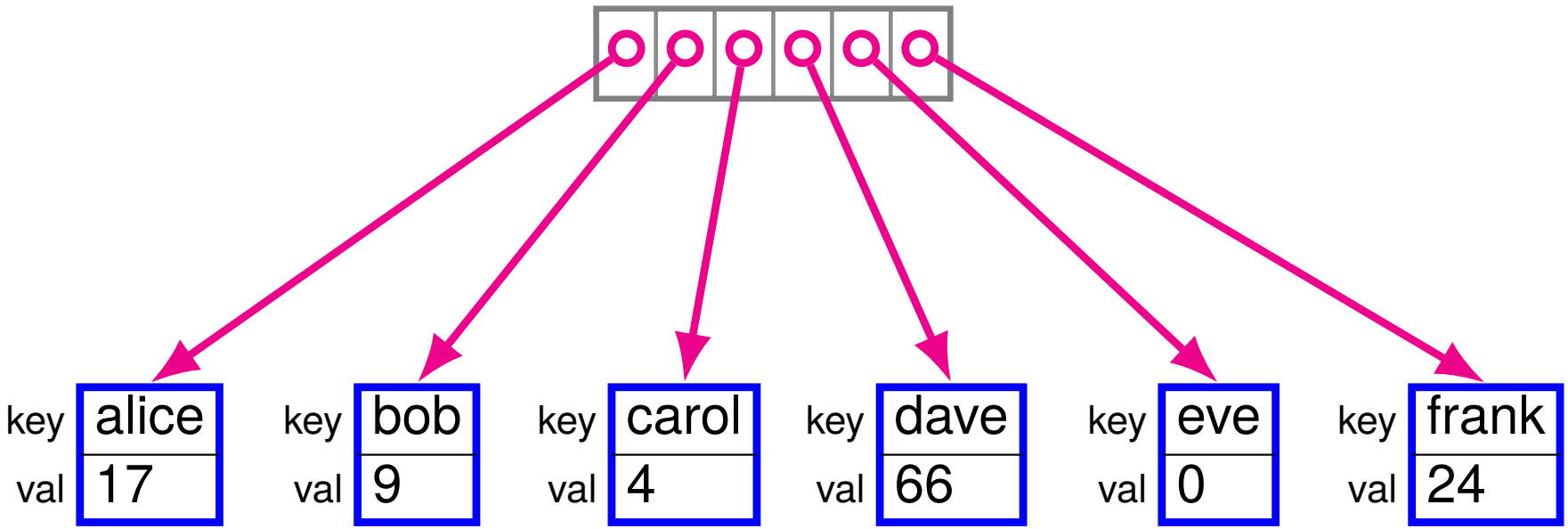
A data structure for dictionaries

There are many data structures we can use to represent dictionaries:

- A sorted array of key-value pairs
- A linked list of key-value pairs (an *association list*)
- A hash table (next time)
- Etc. We'll see more later in the quarter!

They can all provide the necessary operations, but with different tradeoffs!

A sorted array dictionary



- We look things up by key
- Therefore, vector is sorted by key
- How can we do lookups? (i.e., both `mem?` and `get`)

Sorted array lookup: Binary search

We saw it earlier, but let's recap real quick.

We're showing an array with just the keys.

- You can assume the values are tagging along
- We'll do that a lot, for simplicity

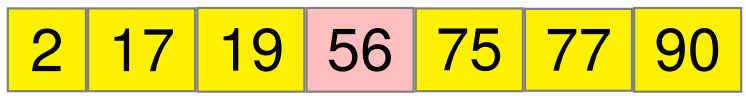
- Suppose we're looking up 75
- Initially, the element could be anywhere in the array

Sorted array lookup: Binary search

We saw it earlier, but let's recap real quick.

We're showing an array with just the keys.

- You can assume the values are tagging along
- We'll do that a lot, for simplicity



- Suppose we're looking up 75
- Initially, the element could be anywhere in the array

- Compare with the middle element

Sorted array lookup: Binary search

We saw it earlier, but let's recap real quick.

We're showing an array with just the keys.

- You can assume the values are tagging along
- We'll do that a lot, for simplicity

- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half

Sorted array lookup: Binary search

We saw it earlier, but let's recap real quick.

We're showing an array with just the keys.

- You can assume the values are tagging along
- We'll do that a lot, for simplicity

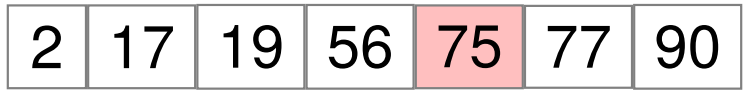
- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half

Sorted array lookup: Binary search

We saw it earlier, but let's recap real quick.

We're showing an array with just the keys.

- You can assume the values are tagging along
- We'll do that a lot, for simplicity

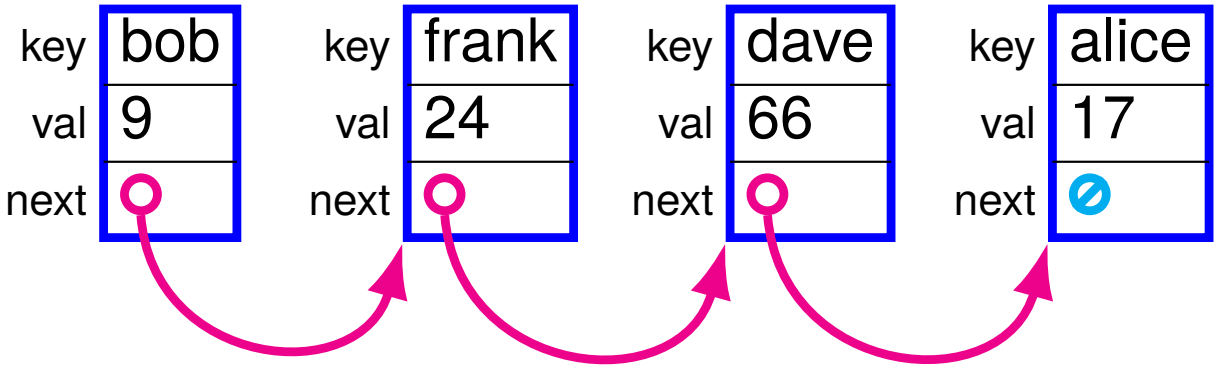


- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half
- This can happen at most $\log_2 n$ times
- Hence $\mathcal{O}(\log n)$. Sweet.

Sorted array insertion

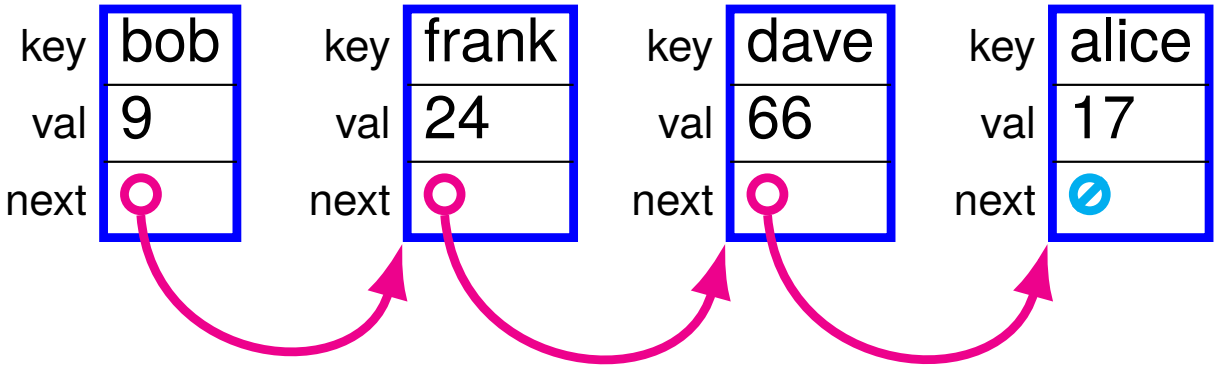
- Inserting into an array requires shifting elements out of the way, to make room for the new one in its correct place
  - Cf., full bookcase analogy
- There may be as many as n elements to move
- Hence insertion is $\mathcal{O}(n)$

An association list dictionary



Linked list of key value pairs.

Association list lookup

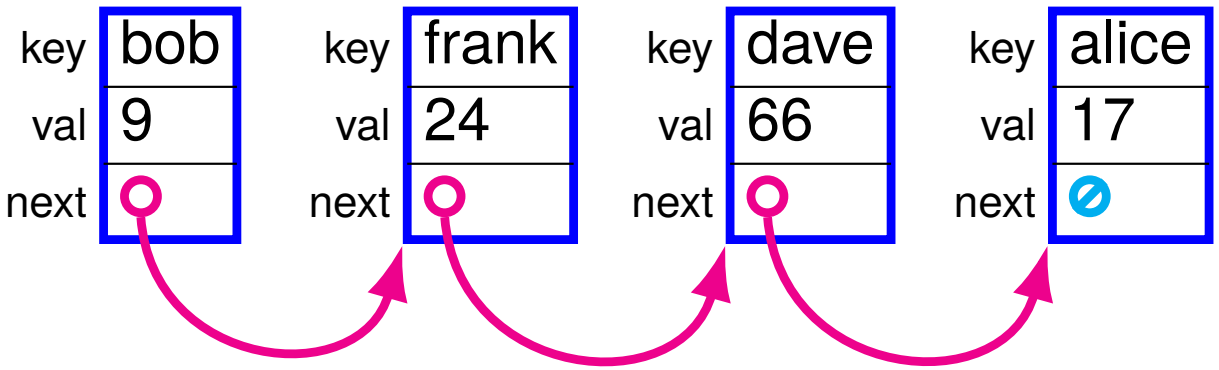


Binary search won't work; linear search is our only option

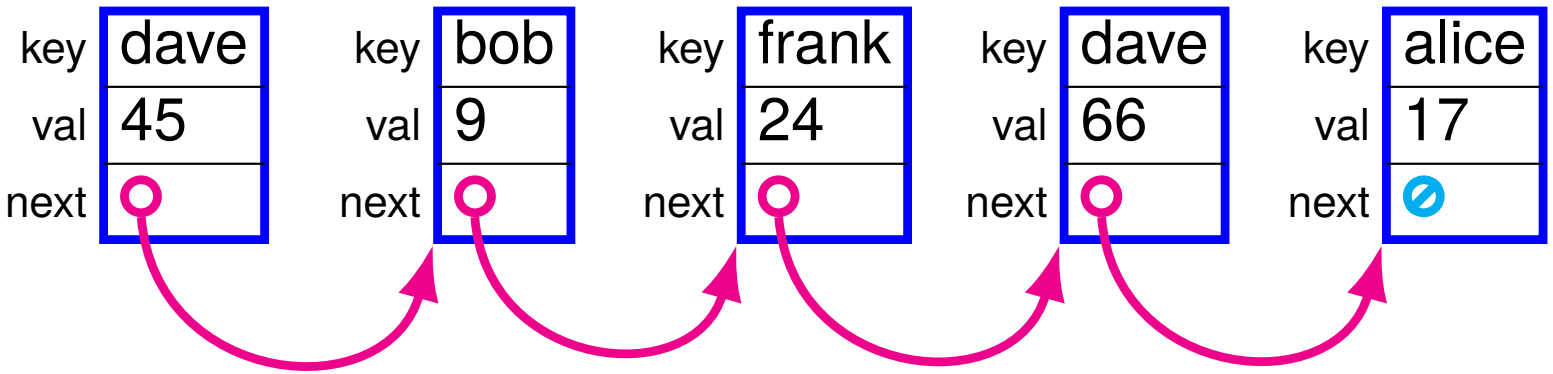
- Elements are not necessarily sorted
- Can't jump to the middle of a linked list anyway

$\rightarrow \mathcal{O}(n)$ worst case

Association list insertion



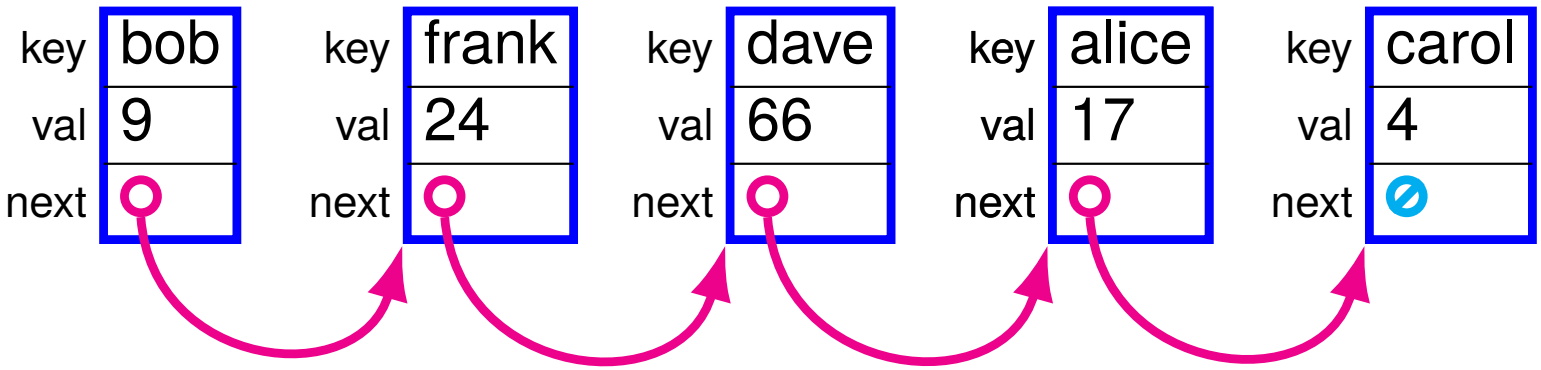
Association list insertion



Inserting at the front ($\mathcal{O}(1)$!) does not work with duplicate keys.

- The same key may exist further down the list!
- But we probably want to either error or update! Oops.

Association list insertion

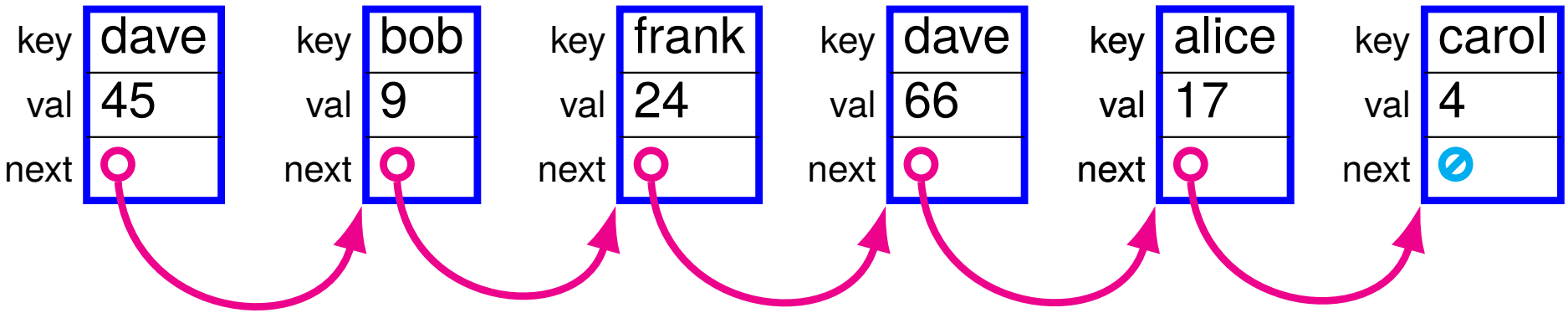


Inserting at the front ($\mathcal{O}(1)$!) does not work with duplicate keys.

- The same key may exist further down the list!
- But we probably want to either error or update! Oops.

Need to go through the entire list, *then* insert (front or back).
$\rightarrow \mathcal{O}(n)$

Association list insertion



Inserting at the front ($\mathcal{O}(1)$!) does not work with duplicate keys.

- The same key may exist further down the list!
- But we probably want to either error or update! Oops.

Need to go through the entire list, *then* insert (front or back).
$\rightarrow \mathcal{O}(n)$

Alternatively, be ok with duplicate keys (and wasted space).

- Always lookup from the start, so can't find later k-v pair
- If you know you'll never insert same key twice, may be ok!

Association lists

Seems like a poor choice complexity-wise.

- $\mathcal{O}(n)$ lookup
- $\mathcal{O}(n)$ insert (or wasting space)

But complexity is not everything!

- Maybe your dictionaries are very small! (small n)
- Simpler than the alternatives! Easier to whip up in a pinch (see homework 3).

We'll have more to say on how to choose data structures later in the quarter.

Stop! Question time!

Before we move on to comparing implementations...

Anything unclear so far?

Anything I missed?

Comparing dictionaries

Multiple data structures following the same ADT interface $\rightarrow$ can swap implementations in and out!

- Cf. `fill_playlist` in Homework 2

Main difference: efficiency $\rightarrow$ let's measure that!

Let's time insertions and lookups as n grows for

- An association list
- A sorted vector dictionary
- A hash table (see next lecture)
- An AVL tree (see later in the quarter)

See `benchmarks.rkt` on Canvas.

Next time: hashing and hash tables